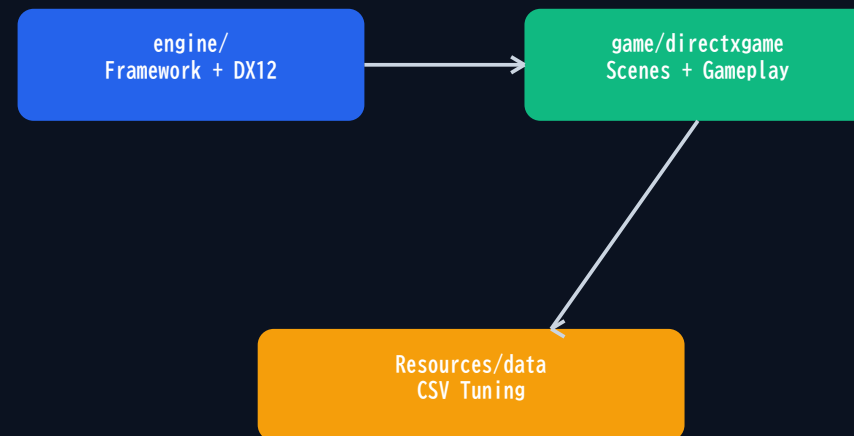


# KuboEngine

## DirectXGame / 3D Survival Shooting

DirectX 12 と C++20 で制作した自作エンジンとゲーム実装。設計、低レイヤリソース管理、データ駆動、検証基盤を中心に説明します。



C++20

DirectX 12

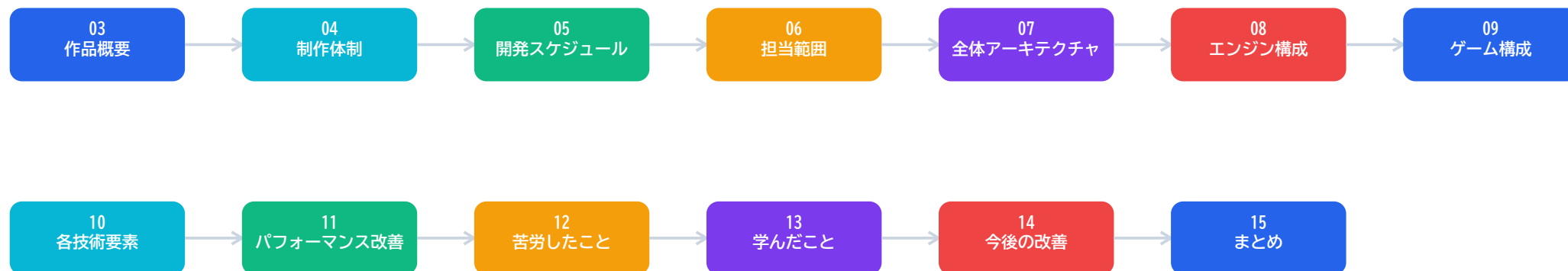
HLSL

CSV Driven

Tests

# 目次

採用担当が見たい規模、担当範囲、技術力、問題解決を順番に迫える構成です。



# 作品概要

敵を倒して EXP を集め、レベルアップで武器や能力を強化していく 3D サバイバルシューティングです。



目的は、ゲームとして遊べる流れを作りながら、DirectX 12 の描画基盤、scene管理、データ調整、負荷対策まで自分で設計することです。

# 制作体制

個人制作として、engine、game、data、tools、testsまで一貫して担当しました。

## engine/

Framework、DirectX、input、audio、resource管理。

## game/

scene、player、enemy、weapon、HUD、effect。

## Resources/data

武器、敵、UI、level-up、resource manifest。

## tools/tests

CPU regression、scene stress、PIX preflight。

# 開発スケジュール

開発は、描画基盤からゲーム実装、データ駆動、リファクタリング、検証へ段階的に進めました。



# 担当範囲

コード全般を担当し、ゲーム実装だけでなく調整・検証の仕組みも作成しました。

## DirectX基盤

frame resource、SRV、pipeline、shader compile  
。

## Gameplay

player、enemy、weapon、level-up、boss。

## UI / Debug

HUD、mini map、debug panel、freeze操作。

## Data

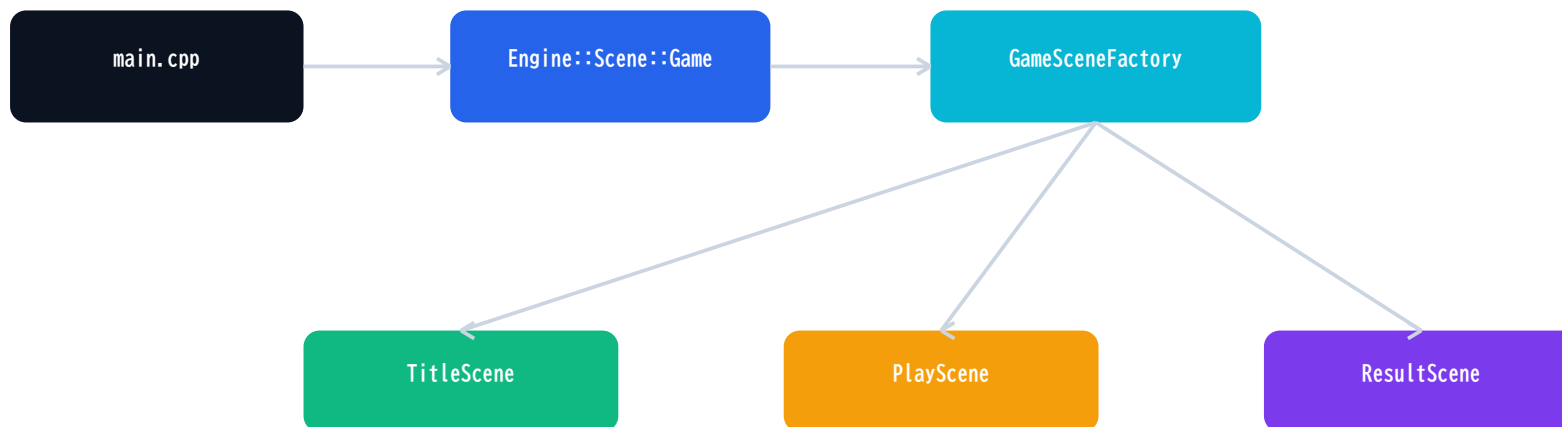
CSV tuning、resource manifest、strict parse。

## Tests

CPU test project、scene transition stress。

# 全体アーキテクチャ

`main.cpp` は起動処理に絞り、GameSceneFactory に scene 生成を委譲します。

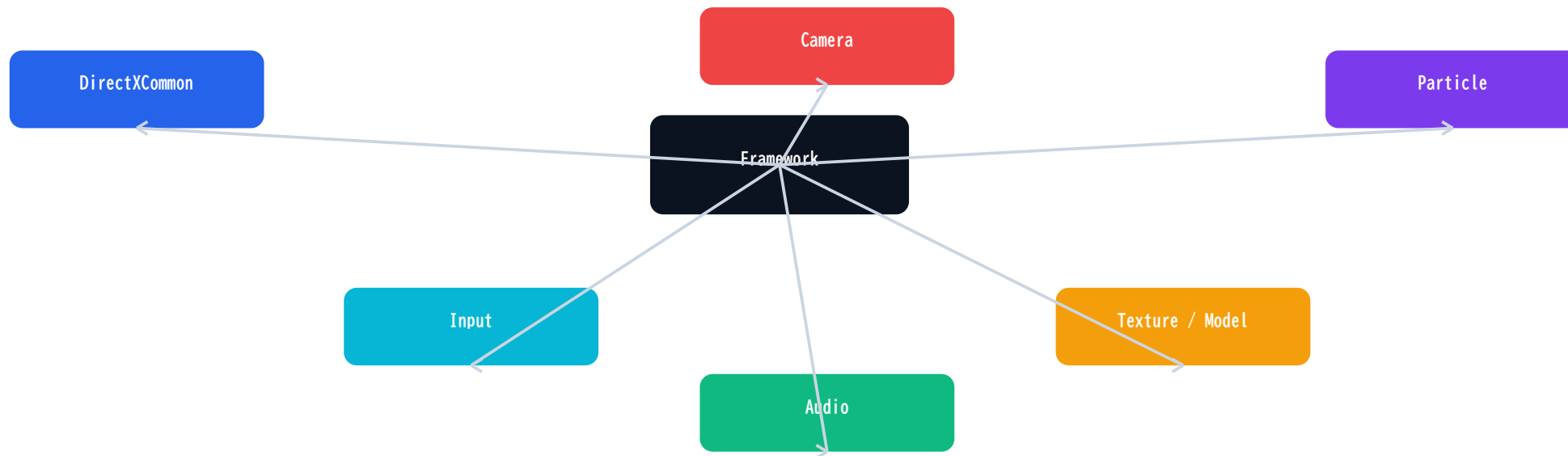


## 設計意図

起動、共通基盤、ゲーム固有sceneの変更理由を分ける。TitleやResultを差し替えてもFrameworkに影響しない構成にした。

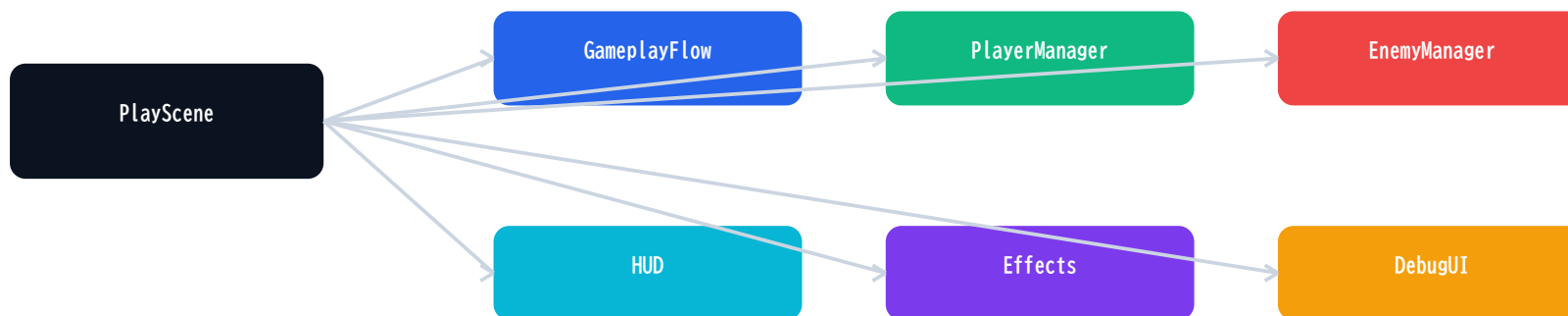
# エンジン構成

`engine/` は DirectX、input、audio、resource、particle、camera、scene infrastructure を担当します。



# ゲーム構成

`PlayScene` をcomposition rootにし、進行、player、enemy、HUD、effect、debugへ分割しました。



大きなscene classへ処理を集めず、変更理由が違う処理を分離しました。ECSのような大きな仕組みではなく、個人制作で追やすい責務分離にしています。

# 各技術要素

設計、描画、データ、衝突、検証を技術要素として整理しました。

## FrameContext

2 frame分のallocator、upload arena、fence、deferred releaseを保持。

## SrvManager

SRV heapを単独所有し、allocate/freeとhigh-watermarkを記録。

## CSV Driven

武器、敵、UI、level-up、resourceをCSVで調整。

## OBB + Spatial

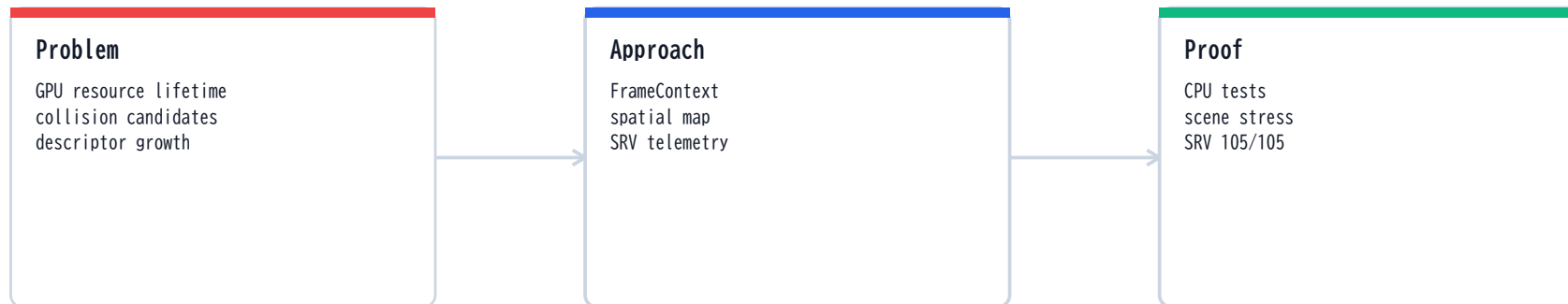
敵や弾の候補を近傍に絞り、回転を含む判定を扱う。

## State Flow

Start、Playing、Boss、Pause、Level Up、Deadを明示。

# パフォーマンス改善

未計測のFPS値は使わず、負荷が増える箇所に対して設計上行った対策を説明します。



# 苦勞したこと

動いた後に見つかる危険箇所を、原因と修正方針に分けて整理しました。

## GPU寿命

問題: GPU参照中のupload resourceを早く解放する危険  
対応: FrameContextとdeferred release

## Scene遷移

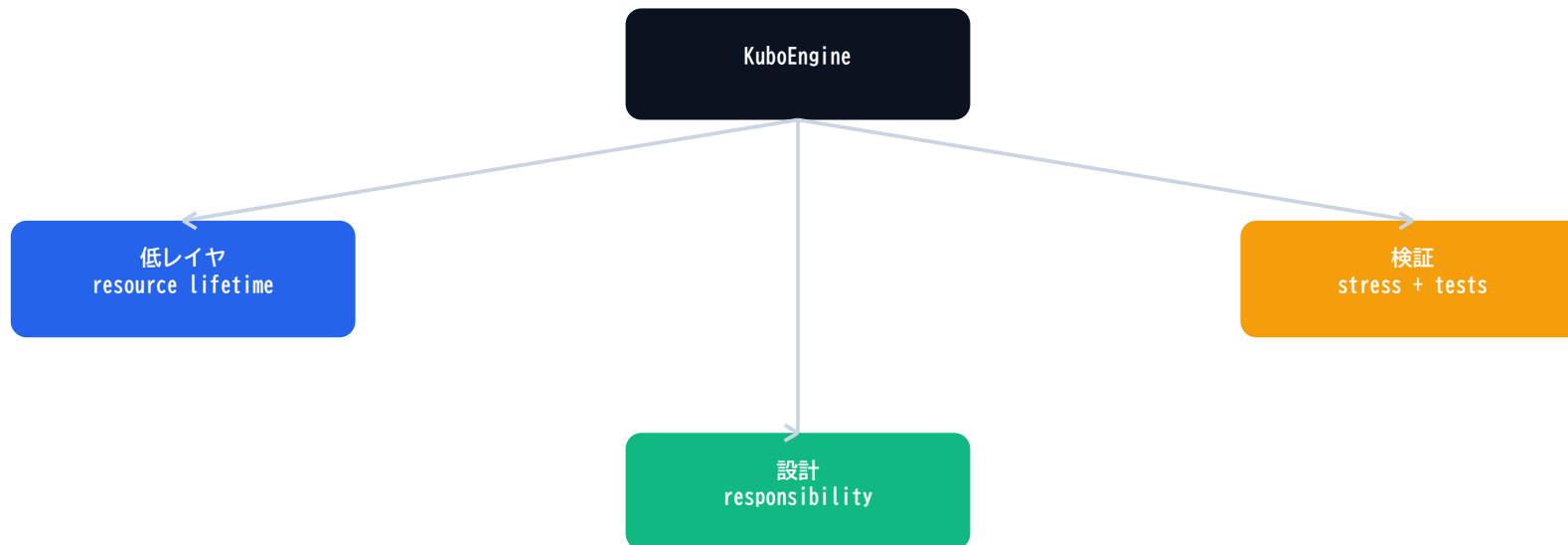
問題: descriptorやtextureが増え続ける危険  
対応: scene stressとSRV high-watermark

## 調整値

問題: 武器や敵の値がコードに散らばる  
対応: CSV化とstrict parse

# 学んだこと

描画APIを使うだけでなく、所有権、調整、検証まで設計する必要があると学びました。



# 今後の改善

現在の設計を土台に、計測と非同期処理を強化します。



# まとめ

KuboEngineは、技術、設計、問題解決、実装、検証を一通り扱った個人制作です。

